

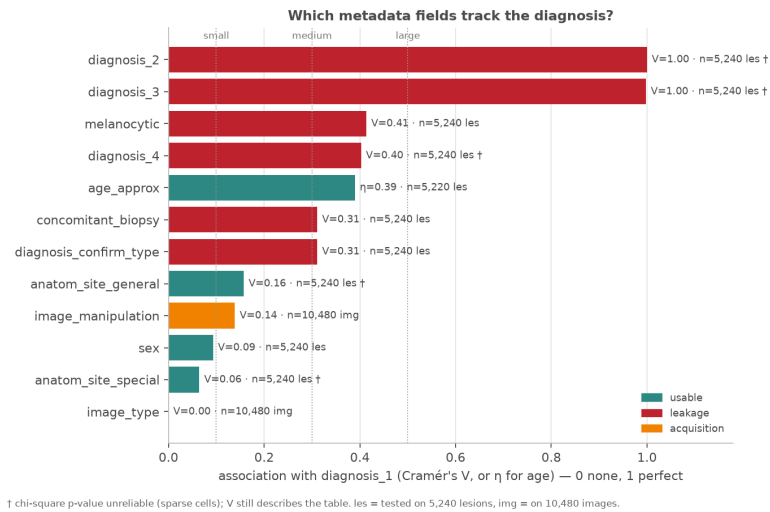
Session 2 homework — Extended EDA & data pipeline (MILK10k)

Stanislaus Lattorff · Computer Vision & Speech Recognition, EADA · September 2026 **Code:** <https://github.com/stansilaisijfnep/milk10k-computer-vision> — `src/milk10k/eda2.py` (Parts 1–2), `pipeline.py` (3–4), `viz.py` (5). Running `python scripts/run_session2.py` recreates every figure and number here.

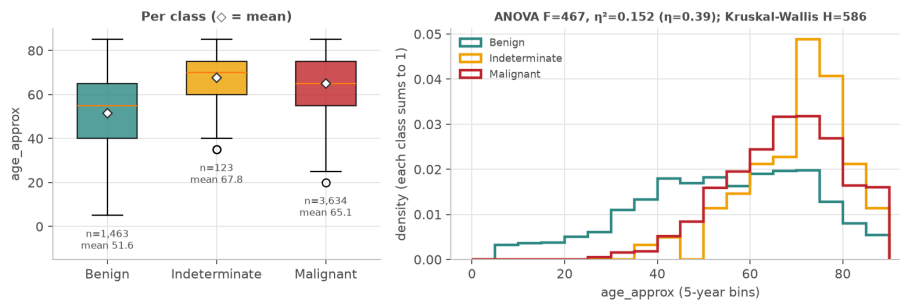
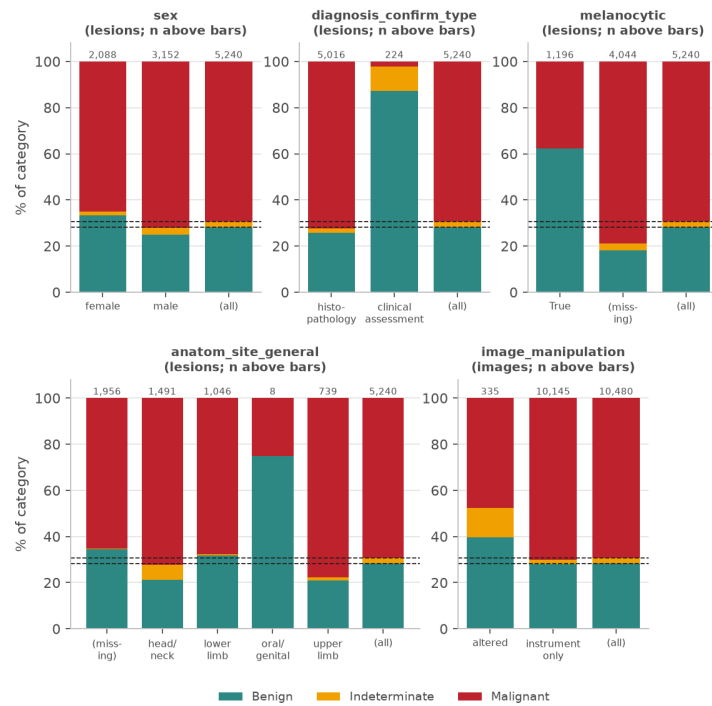
Part 1 — Which metadata fields relate to the diagnosis?

How I approached it

- Each lesion is photographed twice (dermoscopic + clinical), so the 10,480 rows are really 5,240 lesions. From what I understood, testing on the image rows would count every lesion twice, so I ran the tests on **lesions**. Only `image_type` and `image_manipulation` differ between the two photos, so those two I tested on images.
- Categorical fields: **chi-square + Cramér's V**. Age: **ANOVA + η** , with Kruskal–Wallis as a check because age comes in 5-year bins.
- I used V and η because they are both on a 0–1 scale, so I can compare fields directly. With over 5,000 lesions almost every p-value is basically 0, so I think the effect size says more than "significant or not".
- I kept missing values as their own category, since it turned out that *whether* something was filled in says a lot.



Field	My classification	Effect	What I see
diagnosis_2, diagnosis_3	leakage	V = 1.00	more detailed versions of the label itself
melanocytic	leakage	V = 0.41	only filled in for nevi and melanomas
age_approx	usable	η = 0.39	benign avg. 52 y, malignant 65 y, indeterminate 68 y
diagnosis_confirm_type = concomitant_biopsy	leakage	V = 0.31	same variable twice; only 5 of 224 non-biopsied lesions are malignant
anatom_site_general	usable	V = 0.16	most Indeterminate lesions are head/neck; 37 % missing
image_manipulation	acquisition	V = 0.14	edited photos: 13 % Indeterminate vs. 2 % overall
sex, anatom_site_special	usable	V = 0.09 / 0.06	basically no effect
image_type	acquisition	V = 0.00	every lesion has one of each
attribution, copyright_license	constant	—	same value everywhere



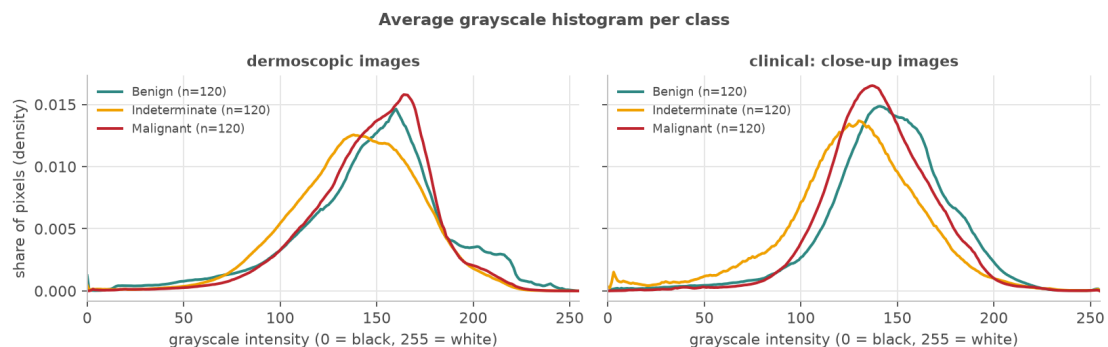
What I take from it

- **Most useful:** age, then body site. **Least useful:** sex, anatom_site_special, image_type and the constant columns.
- **Leakage I would never feed a model:**
 - `diagnosis_2/3/4`: they contain the answer.
 - `diagnosis_confirm_type` / `concomitant_biopsy`: from what I understood, a biopsy only happens when a doctor is already worried, so this tells the model what the doctor thought.
 - `melanocytic`: just knowing it is filled in gives away the lesion type.
- **Bias risk:** `image_manipulation` is about how the photo was made, not the skin, but it still shifts the classes. I think a model could learn "edited photo = indeterminate". Age and site are fine to use, but I would be careful the model doesn't just learn "old + face = malignant".

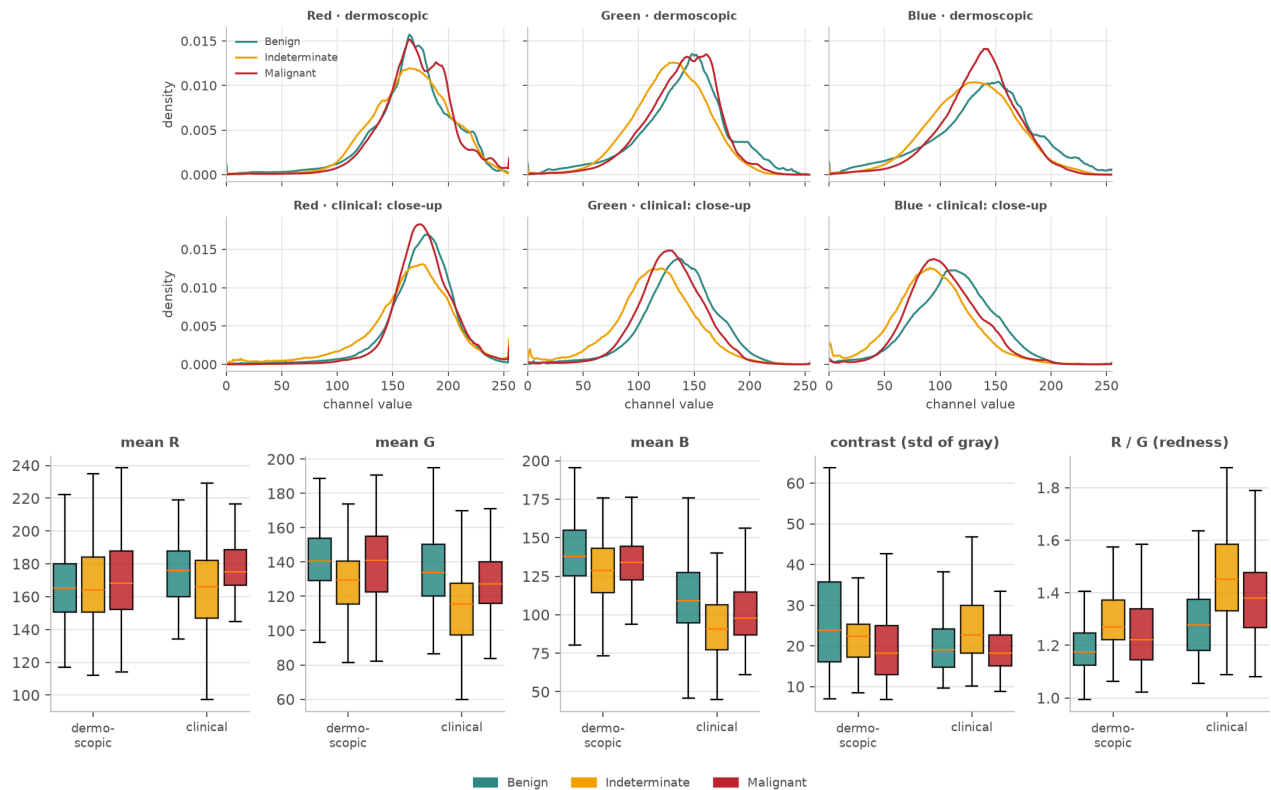
Part 2 – Colour and histograms per class

Setup

- 120 lesions per class (Indeterminate only has 123) and both photos of each, so 720 images at full size. Balanced, so class imbalance can't explain any difference.
- I split dermoscopic and clinical photos because they look very different. Pooling them made the class differences smaller for every feature I measured.



Average per-channel histogram per class



Per image (mean $\pm$ sd)	Dermoscopic: Benign	Indet.	Malignant	Clinical: Benign	Indet.	Malignant
brightness (mean gray)	148 $\pm$ 22	140 $\pm$ 22	148 $\pm$ 20	145 $\pm$ 21	127 $\pm$ 22	140 $\pm$ 19
mean G	141 $\pm$ 22	128 $\pm$ 20	139 $\pm$ 20	136 $\pm$ 23	114 $\pm$ 23	129 $\pm$ 20
redness (R / G)	1.2 $\pm$ 0.1	1.3 $\pm$ 0.1	1.2 $\pm$ 0.1	1.3 $\pm$ 0.2	1.5 $\pm$ 0.2	1.4 $\pm$ 0.2
contrast (std gray)	28 $\pm$ 15	23 $\pm$ 9	21 $\pm$ 11	21 $\pm$ 10	26 $\pm$ 12	19 $\pm$ 6

What I noticed

- Indeterminate images are darker and redder, mostly in the clinical photos.
- Benign dermoscopic images have more bright pixels and more contrast.
- Benign and Malignant look very similar in every channel.

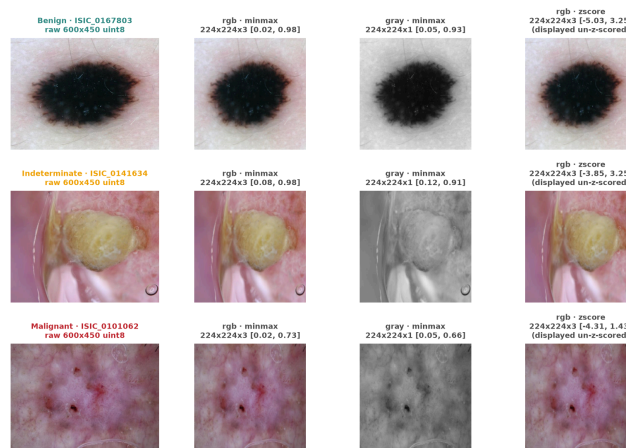
But I don't think it's (only) the lesion

- In my sample, Indeterminate lesions are **80 % on the head/neck** (others 20–28 %), **15 % very fair skin** (others ≤ 1 %) and **17.5 % edited photos** (others 2–5 %). That sounds like sun-damaged facial skin photographed differently.
- 5 of the 120 Indeterminate clinical photos have large dark backgrounds. That's where the spike at 0 in the histogram comes from.
- I couldn't check clinic or camera, because `attribution` is the same for every image.

Is colour alone enough? I don't think so.

- No colour feature explains more than 15 % of its variance by class.
- A simple classifier on 9 colour stats gets **0.47 balanced accuracy on dermoscopic** and **0.56 on clinical** photos (chance is 0.33). That's better than guessing, but not useful, and it's best exactly where the site/skin-type bias is strongest.
- So I think the project needs a model that looks at **shapes and textures** (borders, patterns), i.e. a CNN, with age and site added carefully.

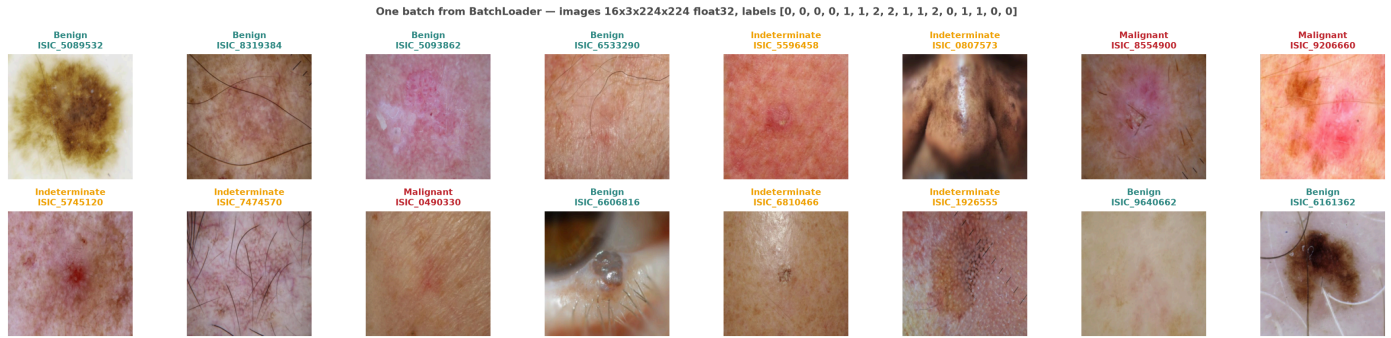
Parts 3–5 — The pipeline



Preprocessing (Part 3)

- `process_image(...)` takes a path, id, PIL image or array. It resizes, converts the colour space (RGB, gray, HSV, Lab) and normalises.
- It returns the array plus a record of what it did: shape, value range and each step.

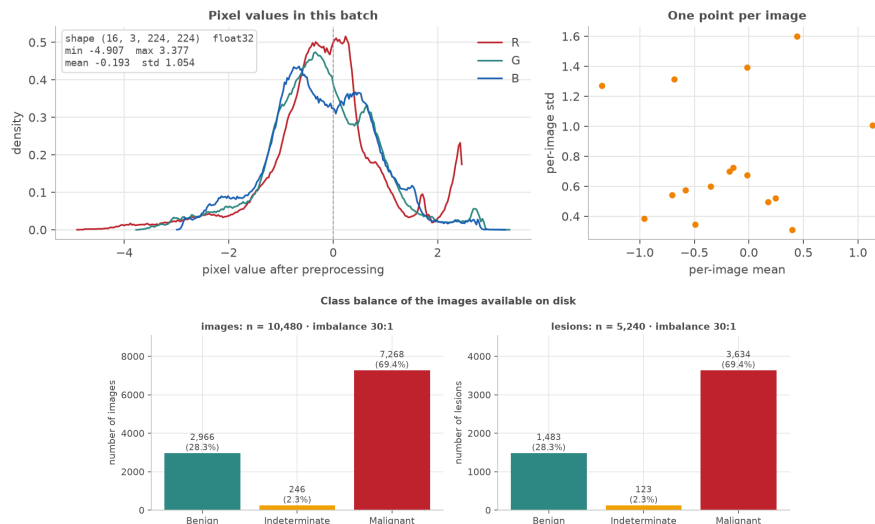
- I chose min-max ($x / 255$) as the default. It needs no statistics from the data, so nothing leaks between train and test. It also keeps brightness differences, which per-image scaling would erase.
- Z-score is optional and uses the mean/std from the **training split only**.
- `process_batch(...)` runs the same thing over many images. If a file is broken or missing, it skips it and says why instead of crashing. I tested it with a cut-off JPEG, a text file and a missing id.



Loader (Part 4)

```
BatchLoader(metadata, img_dir=None, *, batch_size=32, label_col="diagnosis_1",
            classes=("Benign", "Indeterminate", "Malignant"), shuffle=False, seed=42,
            drop_last=False, channels_first=True, **process_image_options)
len(loader)      -> batches per epoch      loader.n_missing / .missing_ids -> rows with no file on disk
for batch in loader:
    batch.images      # float32 (B, 3, 224, 224)      batch.labels      # int64 (B,) class numbers
    batch.label_names # ["Malignant", ...]            batch.ids          # isic_id per image
    batch.skipped     # [(isic_id, reason), ...] for files that failed
```

- It only uses images that are actually on disk. It checks the folder once instead of every file separately.



Visualiser (Part 5) and what it showed me

- The grid above is a real batch from the loader. The plotting function undoes the normalisation first, otherwise the images would look wrong.
- The batch summary confirms z-scoring worked (mean -0.19 , std 1.05). It also shows red pixels maxing out at 255, which I didn't expect.
- **The classes are very imbalanced (30:1)**. A model that always says "Malignant" would already be 69 % accurate. So I think later I'll need class weights or a weighted sampler, and should look at balanced accuracy / per-class recall instead of plain accuracy.

Why I built it this way

- Each piece builds on the one before instead of copying code: `process_image` → `process_batch` → `BatchLoader` → `viz`.
- `process_image` reuses the project's existing resize function, and a test checks the output is identical. This way analysis and training can't end up preprocessing differently.
- Every option has a default, and every result says what was done to it. I wanted future milestones to just call these functions without re-reading the code.